

JavaScript

Chapitre 02

Javascript dans le navigateur

Préambule

L'idée derrière l'utilisation de Javascript dans une page web, c'est de pouvoir modifier dynamiquement celle-ci.

Cela pose les questions suivantes :

- finalement, qu'est-ce qu'une page web ?
- Comment est-elle représentée ? Implantée ?
- Que peut-on faire avec Javascript dans cette page ?



THE HTML PRESENTATION FRAMEWORK



Create Stunning Presentations on the Web

JavaScript et le navigateur

JavaScript dans le navigateur

Architecture d'exécution de Javascript dans le navigateur

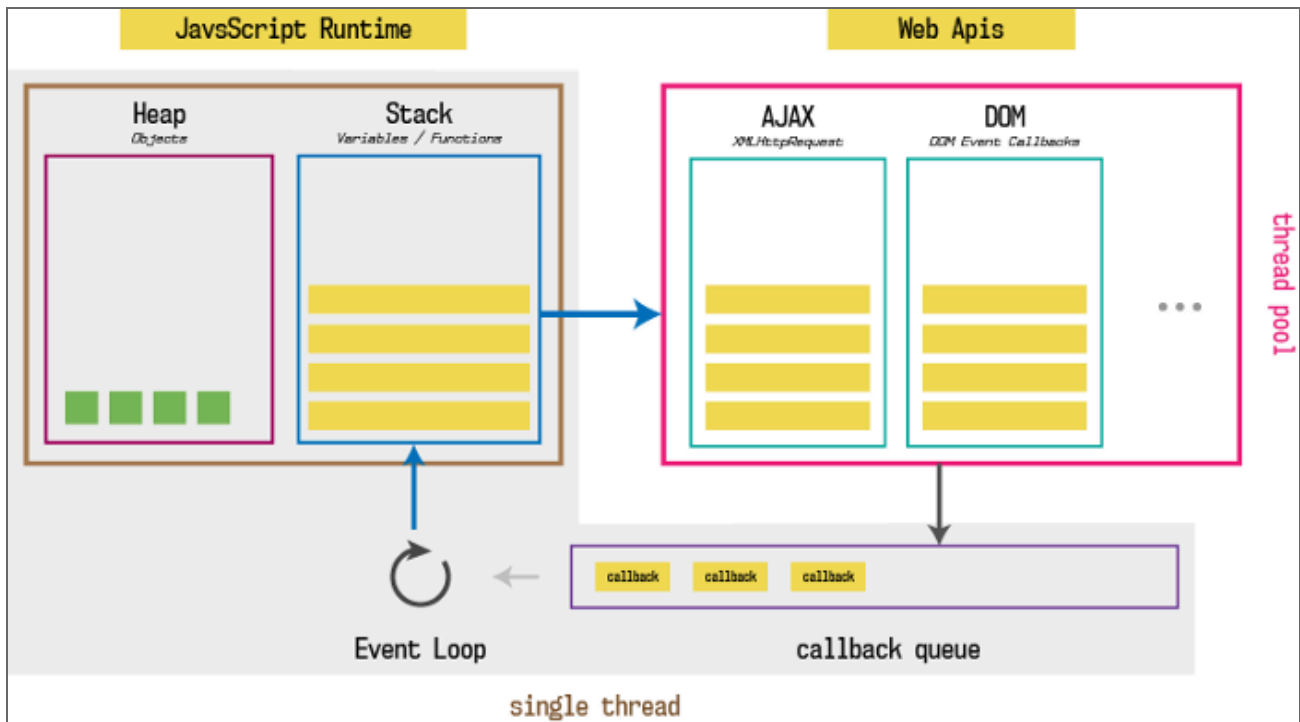
Moteur Javascript

Pour interpréter le Javascript, les navigateurs web utilisent un moteur Javascript (V8, SpiderMonkey, JScript, etc.). Celui-ci repose sur :

- un stockage mémoire (Heap)
- une pile d'exécution (Stack)
- une file d'attente de tâches (callback queue)
- un gestionnaire d'événements (Event Loop)

Web API

En complément du moteur Javascript, les navigateur embarquent des web APIs. Ce sont ces dernières qui permettent de manipuler les éléments utilisés dans le navigateur comme la page html (**DOM**), les évènements (**Event**, **EventListener**, etc.), les requêtes http (**fetch**), etc.



Architecture d'exécution de Javascript dans le navigateur (source: <https://medium.com/jspoint/how-javascript-works-in-browser-and-node-ab7d0d09ac2f>)

pour aller plus loin :

<https://medium.com/@gemma.stiles/understanding-the-javascript-runtime-environment-4dd8f52f6fca>

[https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side web APIs/Introduction](https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Introduction)

<https://www.axopen.com/blog/2020/08/javascript-callback-queue-event-loop/>

Le DOM

Javascript dans le navigateur

Qu'est-ce que le DOM

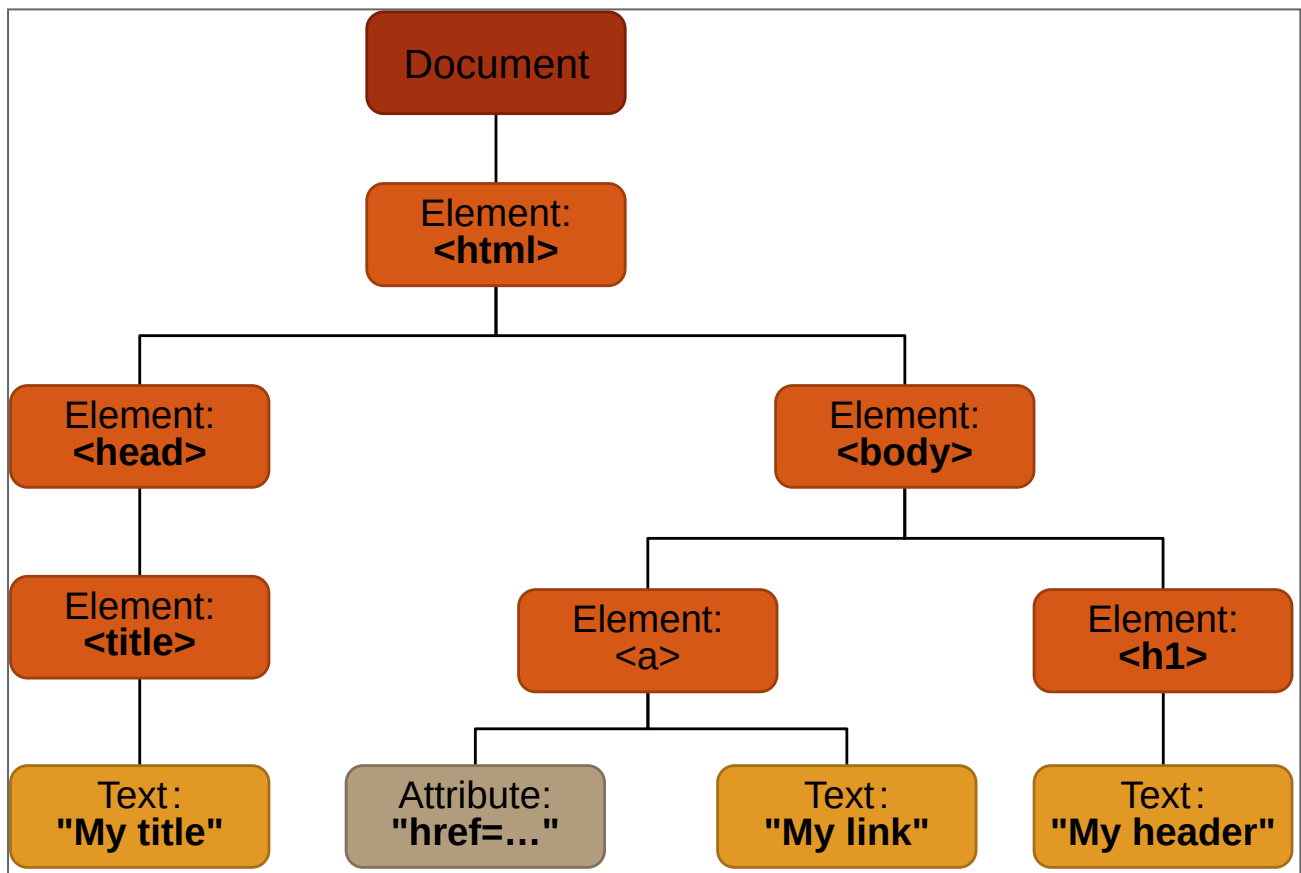
Le DOM (Document Object Model) représente la structure d'une page HTML.

C'est un arbre dont le nœud racine est le **document**.

Celui-ci n'a qu'un seul fils, le nœud **html**.

Chaque nœud est un objet **Node** qui possède deux propriétés :

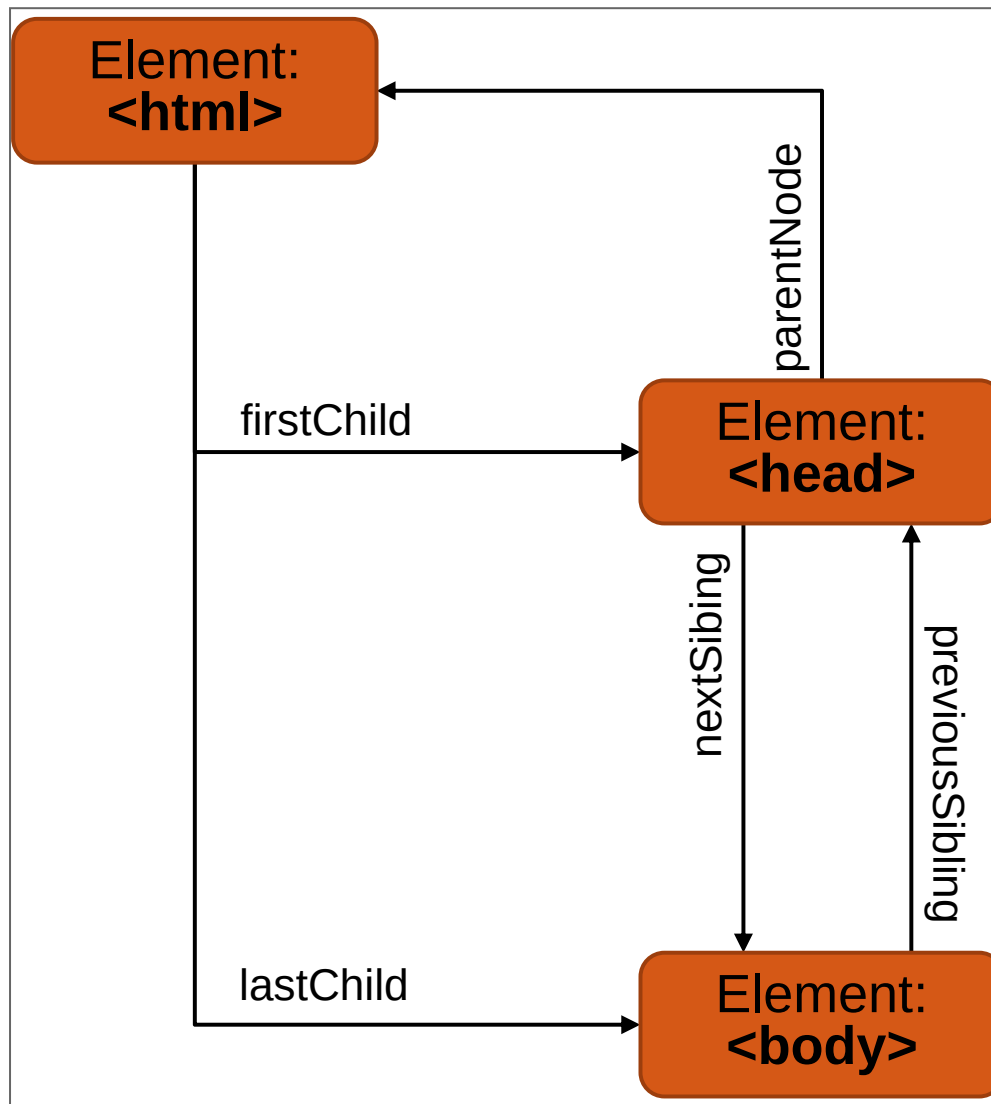
- **parentNode**
- et **childNodes**.



Représentation simplifiée du DOM

Relations entre éléments

Le DOM étant représenté comme un arbre, les nœuds du DOM possèdent des relations de parentés.



Représentation simplifiée du DOM

Manipuler le DOM

Javascript dans le navigateur

Les méthodes GET

Le DOM fournit des méthodes pour accéder rapidement à des éléments HTML spécifiques.

`document.getElementById`

Renvoie une référence à un élément identifié par son id.

```
var content=document.getElementById('content');
```

`document.getElementsByClassName`

Renvoie un objet de type tableau de tous les éléments enfants qui ont tous les noms de classe donnés.

```
var news=document.getElementsByClassName('news');
```

`document.getElementsByTagName`

Retourne une collection des éléments qui ont un nom de tag donné.

```
var paragraphs=document.getElementsByTagName('p');
```

Les sélecteurs

`document.getElementById`,
`document.getElementsByClassName`, et
`document.getElementsByTagName` sont utiles mais
le **DOM X** fournit une méthode plus générale pour
récupérer un ou plusieurs éléments à partir d'un
sélecteur (identique à celui utilisé pour les CSS).

`document.querySelector`

Retourne le premier élément dans le document
correspondant au sélecteur ou groupe de sélecteurs,
spécifié ou `null` si aucune correspondance n'est
trouvée.

```
var content=document.querySelector('#main-content');
```

`document.querySelectorAll`

Renvoie la liste des éléments dans le document (en
partant du premier niveau du html et ordonné comme les
nœuds du document) qui correspondent au groupe de
sélecteurs passés en paramètre. L'objet retourné est une
collection de nœuds de type **NodeList**.

```
var news=document.querySelectorAll('.news');
```

Création de nœuds

Le DOM fournit à travers l'objet **document** des méthodes pour créer de nouveaux nœuds **createElement**, **createTextNode**, **createAttribute**

L'objet **node** quant à lui fournit les méthodes nécessaires pour attacher les nœuds nouvellement créés : **appendChild**

```
// create a new div element
var newDiv=document.createElement("div");

// and give it some content
var newContent=document.createTextNode("Hi there and greetings!");

// add the text node to the newly created div
newDiv.appendChild(newContent);
```

L'objet **node** fournit également le nécessaire pour supprimer, remplacer, tester un nœud.

voir **removeChild**, **replaceChild**, **insertBefore**, **hasChildNode** et **append**.

Modifier un nœud

Chaque élément du DOM possède deux propriétés : **textContent** et **innerHTML**.

textContent

Permet d'accéder et de changer le contenu de l'élément. Le contenu HTML ne conduira pas à la modification du DOM.

```
var title=document.querySelector('h1:first-of-type');  
title.textContent="New title";
```

innerHTML

Permet d'accéder et de changer le contenu de l'élément. La présence de balise HTML conduira à mettre à jour le DOM.

```
var loading=document.querySelector('#loadingPage');  
loading.innerHTML("<span>done.</span>");
```

Comme vous pouvez le remarquer, il est possible de créer des nouveaux éléments dans le DOM en utilisant la méthode **createElement** ou la propriété **innerHTML**. Il est en principe préférable d'utiliser la première méthode.

aller plus loin :

<https://www.javascripttutorial.net/javascript-dom/javascript-innerhtml-vs-createelement/>

<https://stackoverflow.com/questions/2946656/advantages-of-createelement-over-innerhtml/29837285>

<https://www.measurethat.net/Benchmarks/Show/10096/vs-clonenode-vs-innerhtml>

Les événements

Javascript dans le navigateur

Capture et bouillonnement (Capturing / Bubbling)

La propagation des événements dans le DOM comprend 3 phases :

.. capture (**capturing**)

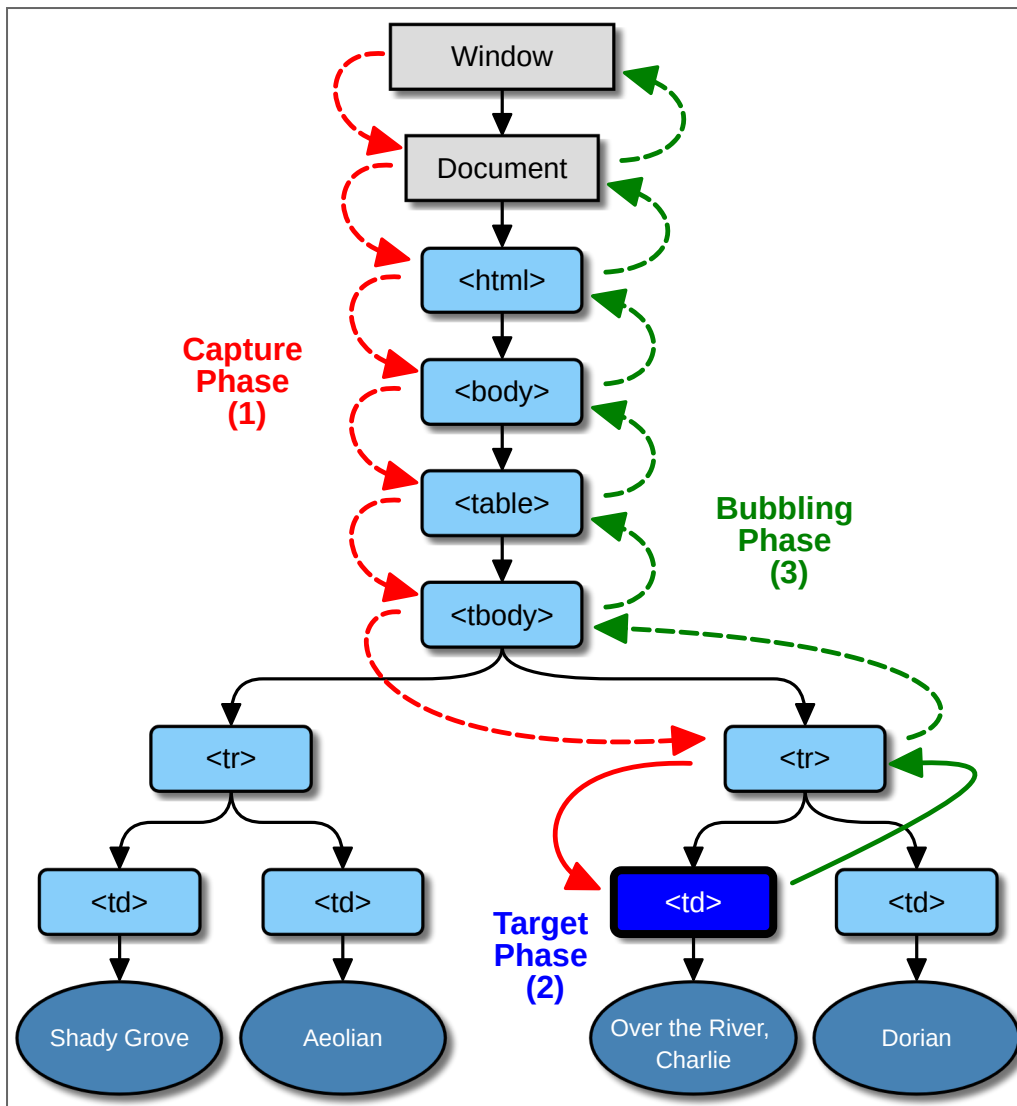
l'événement se propage du nœud **document** au nœud cible exclu.

?. cible (**target**)

l'événement atteint le nœud cible.

}. bouillonnement (**bubbling**)

l'événement se propage du nœud cible exclu au nœud **document**.



Représentation simplifiée du DOM (source : <https://www.w3.org/TR/DOM-Level-3-Events/>)

Gérer les événements

Chaque élément du DOM possède une méthode appelée **addEventListener** qui permet de lui associer un gestionnaire d'événement (handler) sur un événement particulier.

target.addEventListener(type, listener[, options])

```
function handler(e) {  
    console.log("click");  
}  
  
var info=document.querySelector("#info");  
info.addEventListener("click",handler);
```

Pour supprimer le gestionnaire d'événement, il suffit d'utiliser la méthode **removeEventListener**.
target.removeEventListener (type, listener [, options])

Séparer les événements du contenu

Il est aujourd'hui déconseillé d'utiliser les gestionnaires d'événement du **DOM 0** à cause des problèmes d'occlusion que cela peut produire.

```
<a href="#"onclick="foo1()" ></a>
```

Il est préférable d'utiliser l'événement **load** de l'objet **window**. On a aussi du coup la garantie que toute la page est chargée.

```
function handler(e) {  
    console.log("click");  
}  
  
function initHandler() {  
    var info=document.querySelector("#info");  
    info.addEventListener("click",handler);  
}  
  
window.addEventListener("load",initHandler);
```

Intégration de JavaScript dans une page web

Javascript dans le navigateur

Intégration du code

Il existe deux méthodes pour intégrer du code Javascript dans une page web.

Dans les deux cas la balise peut être placée soit dans la section `<head>` de la page html, soit à la fin du document avant la balise fermante `</body>`. Cette deuxième solution est souvent utilisée mais n'est pas nécessairement le meilleur choix.

Directement dans la page

En ajoutant le code directement dans la page dans la balise `<script>`.

```
<script type="text/javascript">  
  //mon code Javascript ici  
</script>
```

En passant par un fichier externe

En faisant référence au sein de la page web à un fichier qui contient le code JavaScript.

```
<script src="monscript.js"></script>
```

aller plus loin : **[https://developer.mozilla.org/en-US/docs/Learn/HTML/Howto/Use JavaScript within a](https://developer.mozilla.org/en-US/docs/Learn/HTML/Howto/Use_JavaScript_within_a)**

Speaker notes

dernière mise à jour le 02/09/2024

JAVASCRIPT DANS LE NAVIGATEUR / B. Desbenoit